

History Of Python

The history of Python is a fascinating journey that transforms a solo Christmas holiday hobby project into the world's most dominant programming language.

Who Invented Python?

- Creator: Guido van Rossum
- Started: 1989, at Centrum Wiskunde & Informatica (CWI), Netherlands
- First Release: 1991
- Reason: Started as a hobby project during Christmas holidays
- Predecessor: Inspired by the ABC language, but Guido improved it by fixing its limitations

Why is it called Python?

- Named after the BBC comedy series “Monty Python’s Flying Circus”
- Guido wanted a name that was short, unique, and a little mysterious
- He served as Python’s BDFL (Benevolent Dictator for Life) until 2018
- Worked at Google and later at Dropbox

Evolution of Python

When first released, Python already supported:

- Classes with inheritance
- Core data types
- Exception handling
- Functions

Version 1

Python reached version 1.0 in January 1994. The major new features included in this release were the functional programming tools [lambda](#), [map](#), [filter](#) and [reduce](#). Van Rossum stated that "Python acquired lambda, reduce(), filter() and map(), courtesy of a [Lisp](#) hacker who missed them and submitted working [patches](#)".^[16]

The last version released while Van Rossum was at CWI was Python 1.2. In 1995, Van Rossum continued his work on Python at the [Corporation for National Research Initiatives](#) (CNRI) in [Reston, Virginia](#) from where he released several versions.

By version 1.4, Python had acquired several new features. Notable among these are the Modula-3 inspired [keyword arguments](#) (which are also similar to [Common Lisp](#)'s keyword arguments) and built-in support for [complex numbers](#). Also included is a basic form of [data hiding](#) by [name mangling](#), though this is easily bypassed.^[17]

During Van Rossum's stay at CNRI, he launched the Computer Programming for Everybody (CP4E) initiative, intending to make programming more accessible to more people, with a basic "literacy" in programming languages, similar to the basic English literacy and mathematics skills required by most employers. Python served a central role in this: because of its focus on clean syntax, it was already suitable, and CP4E's goals bore similarities to its predecessor, ABC. The project was funded by [DARPA](#).^[18] As of 2007, the CP4E project is inactive, and while Python attempts to be easily learnable and not too arcane in its syntax and semantics, outreach to non-programmers is not an active concern.^[19]

Version 2

Python 2.0, released October 2000,^[8] introduced [list comprehensions](#), a feature borrowed from the [functional programming](#) languages [SETL](#) and [Haskell](#). Python's syntax for this construct is very similar to Haskell's, apart from Haskell's preference for [punctuation](#) characters and Python's preference for alphabetic keywords. Python 2.0 also introduced a garbage collector able to collect reference cycles.^[8]

Python 2.1 was close to Python 1.6.1, as well as Python 2.0. Its license was renamed [Python Software Foundation License](#). All code, documentation and specifications added, from the time of Python 2.1's alpha release on, is owned by the [Python Software Foundation](#) (PSF), a [nonprofit organization](#) formed in 2001, modeled after the [Apache Software Foundation](#).^[23] The release included a change to the language specification to support nested scopes, like other [statically scoped](#) languages.^[24] (The feature was turned off by default, and not required, until Python 2.2.)

Python 2.2 was released in December 2001;^[25] a major innovation was the unification of Python's types (types written in [C](#)) and classes (types written in Python) into one hierarchy. This single unification made Python's object model purely and consistently object oriented.^[26] Also added were [generators](#) which were inspired by [Icon](#).^[27]



Historic Python logos used on Windows (left) and the Macintosh (center), and the logo used since version 2.5 (right).

Python 2.5 was released in September 2006^[28] and introduced the `with` statement, which encloses a code block within a context manager (for example, acquiring a [lock](#) before the block of code is run

and releasing the lock afterwards, or opening a [file](#) and then closing it), allowing [resource acquisition is initialization](#) (RAII)-like behavior and replacing a common try/finally idiom.^[29]

Python 2.6 was released to coincide with Python 3.0, and included some features from that release, as well as a "warnings" mode that highlighted the use of features that were removed in Python 3.0.^{[30][10]} Similarly, Python 2.7 coincided with and included features from Python 3.1,^[31] which was released on June 26, 2009. Parallel 2.x and 3.x releases then ceased, and Python 2.7 was the last release in the 2.x series.^[32] In November 2014, it was announced that Python 2.7 would be supported until 2020, but users were encouraged to move to Python 3 as soon as possible.^[33] Python 2.7 support ended on January 1, 2020, along with [code freeze](#) of 2.7 development branch. A final release, 2.7.18, occurred on April 20, 2020, and included fixes for critical bugs and release blockers.^[34] This marked the [end-of-life](#) of Python

Version 3

"Python 3" redirects here. For the Israeli missile, see [Python \(missile\)](#) § Python-3.

Python 3.0 (also called "Python 3000" or "Py3K") was released on December 3, 2008.^[9] It was designed to rectify fundamental design flaws in the language – the changes required could not be implemented while retaining full backwards compatibility with the 2.x series, which necessitated a new major version number. The guiding principle of Python 3 was: "reduce feature duplication by removing old ways of doing things".^[36]

Python 3.0 was developed with the same philosophy as in prior versions. However, as Python had accumulated new and redundant ways to program the same task, Python 3.0 had an emphasis on removing duplicative constructs and modules, in keeping with the [Zen of Python](#): "There should be one— and preferably only one —obvious way to do it".

Nonetheless, Python 3.0 remained a [multi-paradigm language](#). Coders could still follow [object-oriented](#), [structured](#), and [functional](#) programming paradigms, among others, but within such broad choices, the details were intended to be more obvious in Python 3.0 than they were in Python 2.x.

Compatibility

Python 3.0 broke backward compatibility, and much Python 2 code does not run unmodified on Python 3.^[37] Python's [dynamic typing](#) combined with the plans to change the semantics of certain methods of dictionaries, for example, made perfect [mechanical translation](#) from Python 2.x to Python 3.0 very difficult. A tool called "`2to3`" does the parts of translation that can be done automatically. At this, `2to3` appeared to be fairly successful, though an early review noted that there were aspects of translation that such a tool would never be able to handle.^[38] Prior to the roll-out of Python 3, projects requiring compatibility with both the 2.x and 3.x series were recommended to have one source (for the 2.x series), and produce releases for the Python 3.x platform using `2to3`. Edits to the Python 3.x code were discouraged for so long as the code needed to run on Python 2.x.^[10] This is no longer recommended; as of 2012 the preferred approach was to create a single code base that can run under both Python 2 and 3 using compatibility modules.[†]